

24CSA212 – Full Stack Frameworks

WORKING WITH MONGODB- INSTALLATION- BASIC CRUD
OPERATIONS - COMMANDS

MongoDB Introduction

- MongoDB is a popular **open-source, NoSQL database** designed for high performance, scalability, and flexibility.
- Unlike traditional relational databases that store data in tables, MongoDB uses a **document-oriented data model**, where data is **stored in** flexible, JSON-like documents called **BSON (Binary JSON)**.
- This allows developers to **store complex data structures easily** and **modify schemas dynamically** without affecting existing data.
- Known for its ability to handle **large volumes of unstructured data**, MongoDB is widely used in modern web and mobile applications, especially in MEAN and MERN stack development, due to its seamless integration with JavaScript-based technologies.

MongoDB download

- In Web browser, give **mongodb download**

Resources – Server – Left pane – windows – scroll down –
MongoDB Community Edition -

Select accordingly

Download

Open the .exe file from downloads - install as a service –
Check the installation of Compass [Compass is a GUI version
of MongoDB]

Open the Desktop icon - connect

Install MongoDB shell

- In browser, give **mongoshell download**
- Open .exe file – extract – open folder- bin – copy path
- In cmd – environment variables – system variables – new- give name as **mongosh** – paste the copied path-ok-ok-ok
- Open the application file of Mongoshell
- Will open a cmd shell- give the command '**mongosh**'
- Will see the details
- cls- clear screen
- exit - to exit the application

MongoDB shell in VS code

- Open VS code – add extension – ‘Mongodb for VS code’ - Install
- Click on the MongoDB icon on the left pane – connect-Right click – Launch mongodb shell
- It will launch mongodb shell
- Give - `cd path of mongosh`
- `.\mongosh $Env:MDB_CONNECTION_STRING;`
- It will prompt to test>
- If its error
- In VS code terminal
& "give the path of the mongosh.exe file"
Try mongosh again

MongoDB shell in VS code

- If its still error

&env:path [Check whether its returning the path of mongosh]

- Else

Set-Alias mondosh "path of mongosh.exe file"[it returns the prompt if successful]

- Try 'mongosh'
- Remember : its temporary. The alias now created will go away when you close VS code or terminal

MongoDB shell in VS code

- To make it permanent, we have to add it to your powershell so that its always available

- In Powershell

`$PROFILE`

It opens a blank notepad file

In Powershell

- `if (!(Test-Path -Path $PROFILE)) { New-Item -Type File -Path $PROFILE -Force }`
- `notepad $PROFILE`

MongoDB shell in VS code

- In the new notepad file that opens, give

Set-Alias mongosh "path of mongosh.exe file"

Save the file and close notepad

Restart VS code or your powershell Terminal

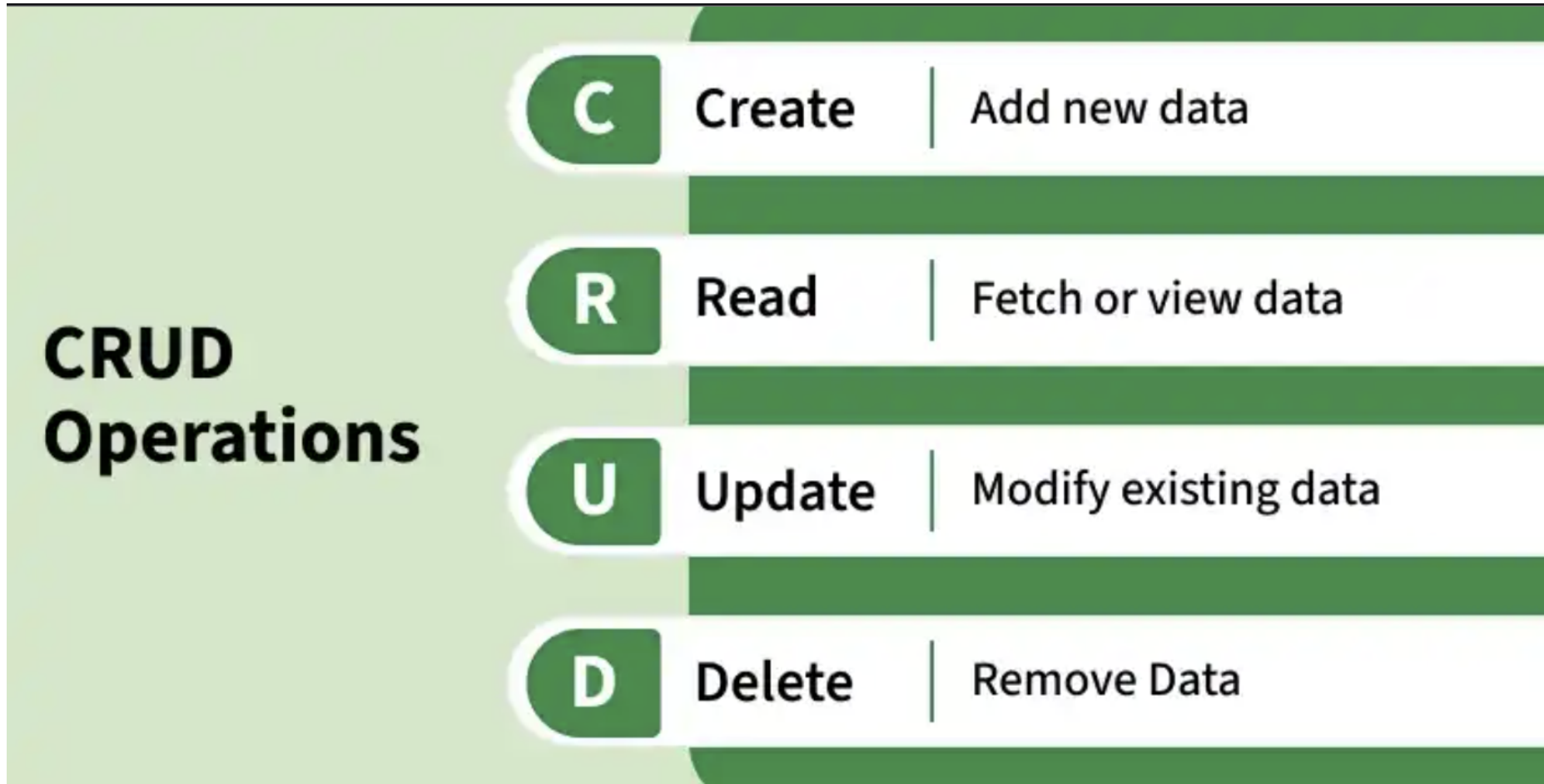
After restart – open VS code – powershell – give command

'mongosh' - it runs

Test by giving **show dbs** [will display the databases available]

MongoDB CRUD Operations

- CRUD operations are fundamental database actions in MongoDB that enable users to insert, read, update, and delete documents within collections.



MongoDB CRUD Operations

1. Create Operations

- Create (insert) operations are used to add new documents to a collection, creating the collection automatically if it does not already exist.
 - Adds new documents to a collection.
 - Automatically creates the collection if missing.
 - Generates a unique `_id` if not provided.
 - Supports inserting single or multiple documents.

Mongosh

- To use a database

use databasename [switched to db admin]

Eg: use school

If we use 'use' with a non-existing db, it will create a new db
But when use 'show' it will not display the newly created db,
because its empty.

To add a collection to the school database

db.createCollection("student")

Try giving 'show dbs' [now it will display school also]

Mongosh

- To drop a database

db.dropDatabase() [will display ok:1 dropped:'school']

To insert one document[record] to the collection inside a db]

use school

db.student.insertOne({name:"abc",age:30,gpa:3.2})

//field: value pair format separated by comma

- To insert many document[record] to the collection inside a db]

db.student.insertMany([

**{name:"xyz",age:32,gpa:4.0},
{name:"cde",age:40,gpa:2.8},
{name:"mno",age:28,gpa:2.8}
])**

//field : value pair format separated by comma, as array

Mongosh

2. Read Operations

- Read operations are used to retrieve or query documents from a MongoDB collection.
- Fetch documents from a collection.
- Support filtering, projection, sorting, and limiting results.
- Used to read and analyze stored data.

To return all documents within a collection

`db.student.find()`

//an id will be automatically added to each document

```
db.student.find().pretty()
```

Mongosh

We are retrieving the details of a single student from the student collection using the `db.collection.findOne()` method.

```
db.student.findOne({name:"xyz"});
```

Mongosh

3. Update Operations

- Update operations are used to modify existing documents in a MongoDB collection based on specified query conditions.
- Update single or multiple matching documents.
- Use update operators like \$set, \$inc, and \$unset.
- Can replace entire documents if required.

We are updating the age of xyz in the student collection using **db.collection.updateOne()** method.

Eg.

```
db.student.updateOne({name:"xyz"},{$set:{age:24}})
```

Mongosh

3. Update Operations

We are updating the year of course in all the documents in the student collection using **db.collection.updateMany()** method.

```
db.student.updateMany({}, {$set: {year: 2020}});
```

We are replacing the complete details of a single student document in the student collection using the **db.student.replaceOne()** method.

```
db.student.replaceOne({name: "Kim"}, {name: "Kim", age: 25, branch: "ECE", mode: "offline", paid: false, amount: 0})
```


Mongosh

4. Delete Operations

- Delete operations are used to remove documents from a MongoDB collection.
- Delete documents based on specific conditions.
- Can remove a single document or multiple documents
- Supports deleting all documents in a collection

We are deleting a document from the student collection using [db.collection.deleteOne\(\)](#) method.

```
db.student.deleteOne({name:"xyz"})
```

Mongosh

4. Delete Operations

We are deleting all the documents from the student collection using [db.collection.deleteMany\(\)](#) method.

```
db.student.deleteMany({})
```

Using Compass

- Select db
- Select the collection – ADD Data – Insert document [a document is pre-generated for you]

`{name:"uvw",age:32,gpa,3.9}`

[if there is some error, click on the format icon on the right end]

To insert many, use the format,

`[name:"vvv",age:24,gpa:3.4},`

`{.....},`

`{.....}])`

Datatypes in MongoDB

- String – a series of characters, can contain spaces or numbers
- Integer – whole numbers only, not decimals
- Double – decimals
- Booleans – true/false
- Date – created as a Date object, e.g. `Date: new Date("yyyy-mm-dd hh:mm:ss")` // `new Date()` will give the current date
- null – store nothing
- Arrays
- Nested documents

Datatypes in MongoDB

- `db.student.insertOne({name:"Larry",
age:32,
gpa:2.8,
fulltime:false,
registerDate:new Date(),
graduationDate:null,
courses:["Biology","Chemistry","Calculus"],
address:{street:"123 xyz",
city:"Amsterdam",
zip:123456}
})`

Sort and Limit documents in MongoDB

- Sorting

`db.student.find().sort({name:1})`

//this will sort the documents in the ascending order of name

//use -1 for sorting in descending order

`db.student.find().sort({gpa:-1})`

// sort the documents in the descending order of gpa

Sort and Limit documents in MongoDB

- Limit the number of documents

db.student.find().limit(1)

//will limit the number of documents to be displayed to 1

Combining sort and limit

db.student.find().sort({gpa:-1}).limit(1)

Using Compass for sorting and limit

- Select options-
- In sort option

{name:1}

// -1 for descending

Enter the limit box for limiting the docs

Filtering

Syntax

.find({query},{projection})

Selecting a particular document

db.student.find({name:"abc"})

*// will act like where class in SQL, select * from students where name="abc"*

To add or combine more filters

db.student.find({gpa:4.0,fulltime:true})

Filtering

Syntax

.find({query},{projection})

db.student.find({}, {name:true})

//Lists all documents, but only name field

//similar to project operation in SQL

// returns id and name

//if you dont want id

db.student.find({}, {_id:false, name:true})

//returns only name field with value

Filtering

To get name and gpa

```
db.student.find({}, {_id:false, name:true,gpa:true})
```

Using Compass

Projection - In options, in projection field, **{name:true}** – find

Filtering – filtering field give **{name:"abc"}**

Update document in MongoDB

Syntax

```
db.student.updateOne({filters},{update})
```

Eg:

```
db.student.updateOne({name:"abc"},{$set:{fullTime:true}})
```

```
db.student.find({name:"abc"})
```

//will show the updated document

```
db.student.updateOne({_id:.....},{$set:{fullTime:false}})
```

//update with _id field

```
db.student.find({_id:.....})
```

To remove a field using Update

```
db.student.updateOne({_id:.....},{$unset:{fullTime:''}})
```

//update with _id field, delete the fullTime field of the particular document

Another eg:

```
db.student.updateOne({name:"abc"},{$unset:{fullTime:''}})
```

```
db.students.find({_id:.....})
```

Update Many

```
db.student.updateMany({},{$set:{fullTime:false}})
```

{ } – indicates updates all documents

```
db.student.find()
```

Use of \$exists

If we want to check whether a particular field exists within the selection criteria

```
db.student.updateMany({fullTime:{$exists:false}},{$set:  
{fullTime:true}})
```

// this will first check whether there is any document which doesn't have fullTime field, if yes, it will update those documents with fullTime:true

```
db.student.find()
```

Update using Compass

In the main page, use the edit(pencil icon) for updating

Trash near the field to delete a particular field

+ near the field will add new field

Can change the type also

Delete using MongoDB

Delete one document

```
db.student.deleteOne({name:"abc"})
```

```
db.student.find()
```

Delete Many Documents

```
db.student.deleteMany({fullTime:false})
```

```
db.student.find()
```

```
db.student.deleteMany({registerDate:{$exists:false}})
```

```
//deletes a document if the registerDate field doesn't exists
```


Exporting and Importing Documents

Export

Main Page – Export – select JSON format- select file location – export-close

Import

Main Page – ADD DATA – import JSON format – select file -import

Comparison Operators

\$ne -> not equal to

```
db.student.find({name:{$ne:"abc"}})
```

//finds every document that is not with name – abc

\$lt -> less than

```
db.student.find({age:{$lt:20}})
```

\$lte-> less than or equal to

```
db.student.find({age:{$lte:27}})
```

Comparison Operators contd...

\$tg -> greater than

```
db.student.find({age:{$gt:20}})
```

\$gte-> greater than or equal to

```
db.student.find({age:{$gte:27}})
```

Combining comparison operators

```
db.student.find({gpa:{$gte:3,$lte:4}})
```

$\text{gpa} \geq 3$ and $\text{gpa} \leq 4$

Comparison Operators contd...

IN Operator

Return all documents which have one of these matching values

db.student.find({name:{\$in:["abc","xyz","mno"]}}) // give the list as
an

array

\$nin -> Not IN

Does the opposite

Logical Operators

\$and - > AND //returns true only when all the conditions are true

```
db.student.find({$and:[{fullTime:true},  
                        {age:{$lte:22}}  
                ]})
```

Similarly

\$or ->OR //returns true when either of the condition is true

\$not ->NOT //complement

db.student.find({age:{\$not:{\$gte:30}}}) // to get when age<30

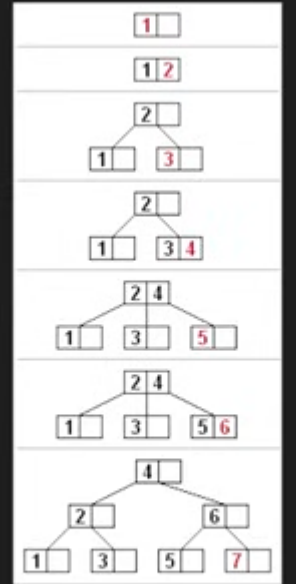
// advantage is that it will return the document even if it is null value

\$nor ->NOR // returns true when all the conditions are false

INDEXES

Indexes

Indexes support the efficient execution of queries in MongoDB. Without indexes, MongoDB must perform a *collection scan*, i.e. scan every document in a collection, to select those documents that match the query statement. If an appropriate index exists for a query, MongoDB can use the index to limit the number of documents it must inspect.



B-Tree

To get the execution status , the number of documents verified, time taken for execution etc.

`db.student.find({name:"abc"}).explain("executionStats")`

Without index, it will go through each and every document. If we have 1000 of documents, the time taken for verification will be immensely large. Hence indexes

INDEXES

To create indexes

db.student.createIndex({name:1} // index will be created in the ascending order of

name. -1 for descending order

On executing, it will show **name_1** which is the index name

db.student.find({name:"abc"}).explain("executionStats")

// will show document examined – only 1, saves time

To show the indexes

db.student.getIndex()

//displays all the indexes, 1 with id(default) and another with name which we created now.

INDEXES

To drop an index

Syntax: `db.student.dropIndex("indexname")`

`db.student.dropIndex("name_1")`

`db.student.getIndex()`

In Compass

Index tab

Create Index – select field – type(ascending/descending) – create index

To Drop – click the Trash icon near the index- select the index name – drop/delete

COLLECTIONS

DATABASE – set of collections

show collections // will show all the collections inside a database

Create collection using parameters

```
db.createCollection("teacher",{capped:true,size:1000000,max:100},  
                           {autoIndexID:false})
```

//teacher- collection name

//capped:true – set a max size limit

//if capped set to true, size should be set in Bytes,

//max:100 – sets the total document size in the collection to 100 numbers

//autoIndexId:false – prevents from creating indexes

Show collections

//lists students and teachers

COLLECTIONS contd...

Create another collection

```
db.createCollection("courses")
```

show collections //shows students, teachers, courses

To drop a collection

```
db.courses.drop()
```